Architectural Audit and Evolutionary Proposal: The Policy of Truth Implementation (DP-OPS-0063 – DP-OPS-0065)

1. Executive Summary: The Transition to Deterministic Governance

The implementation of the "Policy of Truth" (PoT) within the Stela repository represents a paradigmatic shift in software governance, moving from a model of *policing*—where deviations are detected after the fact—to a model of *prevention*, where valid states are constructed by design. This audit analyzes the "Policy of Truth Implementation" sequence (DP-OPS-0063 through DP-OPS-0065), a critical operational phase aimed at resolving the "Constitutional Crisis of Divergence" between the system's governing doctrine and its physical implementation.[1]

The core findings indicate that while the repository has established a "Correct-by-Construction" (CbC) foundation [2], significant friction remains at the interface between automated tooling and human/agent operators. The system has achieved "Backend Self-Awareness"—demonstrated by ops/bin/prune correctly blocking destructive acts on uncommitted evidence [1]—but suffers from "Instructional Drift" in its frontend prompts, which still encourage manual interventions that are now constitutionally illegal.

The repository is currently in a state of "Asymmetric Maturity." The enforcement layer (tools/*) demands absolute literalism, while the guidance layer (docs/) and operational scripts (ops/lib/scripts/*) rely on legacy "State-Guessing" logic. This report synthesizes forensic evidence to propose **DP-OPS-0066**. This proposal aims to close the verification gap, certify all tools against brittleness, and implement a new **Markdown-Based Observability Schema** to make the system actively self-aware.

---

2. Constitutional Analysis: The Canon Surfaces

The governance of the Stela repository relies on a strict hierarchy of truth. This section analyzes the current state of these canon surfaces against "Immutable Infrastructure" principles.[4]

2.1 PoT.md: The Living Constitution

PoT.md serves as the supreme law.[1] Its "Filing Doctrine" asserts that file location dictates function (ops/ for run, docs/ for explain).

- **The "No Invention" Rule:** PoT.md forbids autonomous agents from creating paths not explicitly defined in the constitution.[1] This effectively addresses "Context Sprawl".[5]
- **Enforcement:** tools/verify.sh now actively polices this doctrine, rejecting binary files in docs/ and enforcing the internal structure of opt/.[1]

2.2 TASK.md: From Log to Immutable Contract

TASK.md has shifted from a passive log to an active "Contract".[6]

- **Structural Immutability:** The introduction of ops/src/surfaces/DP.md.tpl [1] and ops/bin/draft [1] implies that TASK.md is no longer *written* by operators but *generated* by the system.
- **Template Hashing:** tools/lint/dp.sh verifies the integrity of the TASK.md structure by comparing its hash against the canonical template.[1] This implements a "Tamper-Evident" design.[7]

2.3 SoP.md and PoW.md: The Dual Ledger

The separation of SoP.md (Narrative History) and PoW.md (Evidence Ledger) establishes a robust "Dual Ledger" pattern.[8]

- **Proof of Work:** PoW.md acts as the "Audit Trail".[9] The pow_policing.yml workflow enforces "Append-Only Integrity," creating a "Chain of Custody" where every claim is linked to a cryptographic proof (RESULTS, OPEN, DUMP artifacts).[1]

---

3. Operational Forensic Review: The Implementation DPs

3.1 DP-OPS-0063: The Stabilization Stress Test

- **Status: BLOCKED** (Safety Guard Violation).[1]
- **Forensic Analysis:** The ops/bin/prune tool correctly blocked a scrub command because the RESULTS file was uncommitted. This "Positive Failure" demonstrates that the system prioritizes "Evidence Integrity" over "Disk Hygiene."[10]

3.2 DP-OPS-0064: Structural Hardening

- **Status: SUCCESS** (Verified Clean Platform State).[1]
- **Forensic Analysis:** This DP successfully migrated the library to opt/_factory and

implemented "Deterministic Compilation" via ops/bin/compile.[1] This resolves the "Recursive Context" problem [11] by firewalling agent definitions from the global context window.

3.3 DP-OPS-0065: Immutable Workflow Adoption

- **Status: SUCCESS**.[1]
- **Forensic Analysis:** By shipping ops/bin/draft and ops/src/surfaces/DP.md.tpl, the system closed the gate on manual TASK.md authoring. The "Structure Hash" check in tools/lint/dp.sh now detects if an operator manually alters the DP boilerplate, forcing the use of the generator tool.

---

4. Toolchain Audit: The Ops Binary Suite

This audit evaluates the ops/bin suite for utility, brittleness, and self-awareness.

4.1 ops/bin/draft (The Architect)

- **Status: Unregistered**.
- **Critical Finding:** While ops/bin/draft exists and functions as the cornerstone of the CbC architecture, it is missing from docs/ops/registry/BINARIES.md.[1] This violates the "No Ghosts" rule; the system has a capability it does not document.

4.2 ops/bin/prune (The Garbage Collector)

- **Status: Brittle**.
- **Critical Finding:** It relies on parsing TASK.md to "guess" the DP_TARGET if not provided. This "State-Guessing" logic contradicts the "Explicit Input" philosophy [12] and creates risk of accidental data loss.

4.3 ops/bin/compile (The Determinism Engine)

- **Status: Robust**.
- **Analysis:** Effectively replaces runtime globbing with compile-time resolution, ensuring "Frozen Context" at session start.

---

5. Script Library Audit: The Logic Layer

5.1 synthesize.sh (Context Engine)

- **Risk:** Must be rigorously audited to ensure it explicitly blacklists the new opt/_factory paths. If it still relies on legacy blacklists, it poses a "Recursive Context Crash" risk.[11]

5.2 agent.sh, skill.sh, task.sh (The Factory)

- **Brittleness:** These scripts likely rely on regex to parse "Draft" files. This "Parsing Brittleness" [12] makes them susceptible to failure if an agent adds an extra space or newline. Moving to structured inputs or stricter templates is required.

---

6. Governance & Verification Audit (tools/*)

6.1 tools/lint/dp.sh

- **Gap:** Currently issues a WARN if the template is missing.[1] This is a "Fail Open" security flaw. It must **Fail Closed** (exit 1) to prevent structural drift.

---

7. Documentation & Prompt Drift: The Human Interface

The "Human Interface" (Prompts) is currently the weakest link.

7.1 The E-PROMPT Divergence

docs/ops/prompts/E-PROMPT-01 through 04 were **not** updated during the immutable workflow transition.[1]

- **The Conflict:** The new toolchain (ops/bin/draft) mandates programmatic generation of TASK.md. The legacy prompts likely instruct the operator to "Copy the DP template and edit manually."
- **The Implication:** An operator following current prompts will violate the system's integrity checks immediately.

---

8. System Self-Awareness: The Observability Graph

"Self-Awareness" in this context is defined as the system's ability to generate a queryable, graph-based record of its own operations, state transitions, and logic checks.[13] We propose moving from "Implicit Awareness" (tools just working) to "Explicit Observability" via the **Markdown-Header Schema**.

The Observability Gap

Currently, tools log to stdout or unstructured text files. This creates "Opaque Execution," where the reasoning behind a tool's success or failure is lost.

The New Standard: Markdown-Header Schema

To achieve true self-awareness, all ops/bin/* and ops/lib/scripts/* tools must adopt a standardized output schema.[1] This transforms logs into a searchable, linkable Knowledge Graph.

Schema Definition:

Each tool execution must append a "Self-Awareness Leaf" to the session log using this structure:

| TOOL_NAME | STATUS:

- **TraceID:** uuid-v4
- **Context:** parent-script-name (e.g., "DP-OPS-0066 Execution")
- **State Check:** "Self-verification completed. Governance limits within bounds."
- **Payload:**Detailed summary of what the tool did in plain text.
- **Provenance:** [Link to source binary or data artifact]

Implementation Tactics:

1. **Front-Matter Metadata:** Use YAML front-matter in log files to allow the "AI OS" to treat them as database records (session_id, compliance_status).
2. **Provenance Leaves:** Tools generate individual leaf files (e.g., logs/trace_8f3a92.md) linked from a central TRUTH.md (or PoW.md).
3. **Tail-f Governance:** "Watchdog" scripts can tail these markdown logs and trigger "Circuit Breakers" if they detect STATUS: FAIL.

---

9. Proposal for Progress: DP-OPS-0066

To solidify the "Correct-by-Construction" architecture and resolve the "Asymmetric Maturity" between tools and prompts, DP-OPS-0066 is proposed.

9.1 Objective

Achieve **System Self-Coherence** by synchronizing the Human Interface (Prompts) with the Machine Interface (Tools), performing a comprehensive anti-brittleness reconfiguration of all scripts, and implementing the **Markdown-Based Observability** layer.

9.2 Scope

- **Tool Reconfiguration Audit:** Review *all* binaries and scripts (ops/bin/*, ops/lib/scripts/*) to certify they are "Explicit Input" driven and free of brittle state-guessing.
- **Prompt Rewrite:** Modify docs/ops/prompts/* to mandate ops/bin/draft usage and

remove legacy authoring instructions.
- **Observability Layer:** Instrument tools to emit the "Markdown-Header Schema" for self-aware logging.
- **Registry Remediation:** Register ops/bin/draft and any other ghost binaries.

9.3 Execution Plan

Phase A: Prompt Synchronization (The Rewrite)

We will rewrite the E-PROMPTs to align with the new immutable workflow. Below are the **proposed contents** for the modified prompts:

Modified docs/ops/prompts/E-PROMPT-01.md (Gatekeeper)

Gatekeeper (Refresh + Audit)

Use when: Auditing worker output before merge.

Attach: DP-RESULTS.md, OPEN, OPEN-PORCELAIN, dump payload, dump manifest.

Rules:

- Refresh state using the attached OPEN and dump artifacts.
- Logic: PoT.md. Structure: TASK.md.
- **Traceability:** Check for "TraceID" and "Provenance" fields in tool outputs.

Steps:

1. AUDIT for PoT.md compliance.
2. Receipts: Verify every changed file has a corresponding diff in RESULTS.
3. **Generated Outputs:** Confirm llms*.txt and manifests/*.md were generated by tooling (check for TraceIDs in their logs), not hand-edited.
4. Drift checks: Ensure no "Context Hazard" violations (e.g., library files in global context).

Output: PASS or FAIL. If FAIL, list specific deviations.

Modified docs/ops/prompts/E-PROMPT-03.md (Architect)

Architect (Refresh + Draft DP)

Use when: Drafting a new DP from a plan.

Attach: OPEN, dump, plan.md.

Rules:

- Refresh state using the attached OPEN and dump artifacts.

- **Immutable Workflow:** Do NOT hand-author the DP structure.
- **Tooling:** You must use ops/bin/draft to generate the DP structure.

Steps:

1. REVIEW for scope and objectives.
2. GENERATE command: Construct the ops/bin/draft command string required to create the DP.
   - Usage: ./ops/bin/draft --id=DP-OPS-XXXX --title="..." --work-branch=...
3. FILL SLOTS: Provide the content for the variable slots (Objective, Scope, Plan) to be passed to the draft tool.
4. **Constraint:** Do not invent file paths. Verify all targets against the dump manifest.

Output: The ops/bin/draft command string and the content for the slots.

Phase B: Comprehensive Tool Reconfiguration

1. **Brittleness Review:** Audit ops/bin/prune, ops/bin/open, and ops/lib/scripts/*.
   - *Action:* Replace all logic that "parses TASK.md to find DP ID" with hard requirements for --dp flags.
   - *Action:* Ensure all regex parsing of headers allows for flexible whitespace.
2. **Explicit Inputs:** Refactor prune to fail if --dp is missing, rather than guessing.

Phase C: Observability Implementation

1. Create ops/lib/scripts/log.sh: A shared library to generate the Markdown Schema output.
   - Function: log_trace "TOOL_NAME" "STATUS" "PAYLOAD" "CONTEXT"
   - Output: Appends structured markdown to storage/logs/trace_[uuid].md.
2. Instrument ops/bin/draft and ops/bin/prune to use log_trace.

Phase D: Hardening

1. **Fail-Closed Linting:** Update tools/lint/dp.sh to exit 1 if DP.md.tpl is missing.
2. **Ghost Busting:** Add ops/bin/draft to docs/ops/registry/BINARIES.md.

---

10. Conclusion

The "Policy of Truth" is only as true as the tools that enforce it. Currently, our tools are "silent workers"—effective but opaque. By implementing **DP-OPS-0066**, we will transform them into "Self-Aware Agents" that produce a traversable graph of their own actions. Combined with the rewritten prompts that enforce the immutable workflow, this will close the final gap between our governance doctrine and operational reality.

Recommendation: Execute DP-OPS-0066 immediately to certify the system as "Self-Aware"

and "Correct-by-Construction."

Works cited

1. Policy of Truth Implementation Plan.md
2. Building Correct-by-Construction Systems with Formal Patterns - Illinois Experts, accessed February 14, 2026, https://experts.illinois.edu/en/publications/building-correct-by-construction-systems-withformal-patterns
3. eBook | Winning the PCB Trifecta via "Correct by Construction" Design Principles - Cadence, accessed February 14, 2026, https://www5.cadence.com/Correct_by_Construction_ebook.html
4. AWS Prescriptive Guidance - Agentic AI patterns and workflows on AWS - AWS Documentation, accessed February 14, 2026, https://docs.aws.amazon.com/pdfs/prescriptive-guidance/latest/agentic-ai-patterns/agentic-ai-patterns.pdf
5. AI Agent Governance: How to Keep Agentic ITOps Workflows Safe | LogicMonitor, accessed February 14, 2026, https://www.logicmonitor.com/blog/ai-agent-governance-agentic-itops-workflows
6. Building Effective AI Agents - Anthropic, accessed February 14, 2026, https://www.anthropic.com/research/building-effective-agents
7. MedBeads: An Agent-Native, Immutable Data Substrate for Trustworthy Medical AI, accessed February 14, 2026, https://arxiv.org/html/2602.01086v1
8. 7 Best Practices for Building a Responsible AI Agent Governance Framework - Electric Mind, accessed February 14, 2026, https://www.electricmind.com/whats-on-our-mind/7-best-practices-for-building-a-responsible-ai-agent-governance-framework
9. Agentic AI Data Governance: Best Practices to Automate Compliance - Acceldata, accessed February 14, 2026, https://www.acceldata.io/blog/enhance-security-and-control-with-agentic-ai-data-governance
10. Custom Cleanup Strategies 101 - JFrog, accessed February 14, 2026, https://jfrog.com/whitepaper/custom-cleanup-strategies-101/
11. AI Agent Architecture: Frameworks, Patterns & Best Practices - Leanware, accessed February 14, 2026, https://www.leanware.co/insights/ai-agent-architecture
12. Design patterns or best practices for shell scripts [closed] - Stack Overflow, accessed February 14, 2026, https://stackoverflow.com/questions/78497/design-patterns-or-best-practices-for-shell-scripts
13. guanyang/antigravity-skills: Empower agents with professional capabilities in specific fields (such as full-stack development, complex logic planning, multimedia processing, etc.) through modular Skills definitions, allowing agents to solve complex problems systematically like human experts. - GitHub, accessed

February 14, 2026, https://github.com/guanyang/antigravity-skills